Exceptions, Exception Handling, Bugs, and Debugging in Python

1. Errors vs Exceptions vs Bugs

Error	Exception	Bug
Mistake in code	Problem during execution	Incorrect program behavior
Often prevents execution	Can be handled	May not produce any error
Example: Missing colon	Divide by zero	Wrong formula

Example:

```
print("Hello"
```

Output: SyntaxError

2. Syntax Errors

A syntax error occurs when Python cannot understand your code.

Example 1

```
if x == 10
   print(x)
```

Output: SyntaxError: expected ':'

Example 2

```
print("Hello)
```

Output: SyntaxError: unterminated string literal

Example 3

```
for i in range(5)
   print(i)
```

Output: SyntaxError

==>These errors must be fixed before the program can run.

3. Runtime Errors (Exceptions)

The program starts executing, but encounters an unexpected situation.

Example

```
a = 10
```

```
b = 0

print(a / b)
```

Output: ZeroDivisionError

4. Logical Errors (Bugs)

The program runs successfully, but gives the wrong answer.

Example

```
length = 10
width = 5

area = 2 * (length + width)

print(area)
```

Output: 30

Expected: 50 (area = length + width)

No exception occurs.

This is a **bug**.

5. Built-in Exceptions

5.1 ZeroDivisionError

```
print(10 / 0)
```

Output: ZeroDivisionError

5.2 NameError

```
print(age)
```

Output: NameError

5.3 TypeError

```
print("10" + 5)
```

Output: TypeError

Correct: print(int("10") + 5)

5.4 ValueError

```
age = int("abc")
```

Output: ValueError

5.5 IndexError

```
numbers = [1,2,3]

print(numbers[5])
```

Output: IndexError

5.6 KeyError

```
student = {
   "name":"Alice"
}

print(student["age"])
```

Output: KeyError

5.7 FileNotFoundError

```
open("marks.txt")
```

Output: FileNotFoundError

5.8 AttributeError

```
name = "Python"

name.append("A")
```

Output: AttributeError

5.9 ImportError

```
from math import xyz
```

Output: ImportError

5.10 ModuleNotFoundError

```
import xyzmodule
```

Output: ModuleNotFoundError

5.11 OverflowError

```
import math

print(math.exp(1000))
```

Output: OverflowError

5.12 MemoryError

```
big = [0] * (10**12)
```

May produce: MemoryError

5.13 RecursionError

```
def f():
    f()

f()
```

Output: RecursionError

6. Basic Exception Handling

Syntax

```
try:
    # risky code
except:
    # recovery code
```

Example

```
try:
    number = int(input("Enter number: "))
    print(100 / number)

except:
    print("Invalid input")
```

7. Catching Specific Exceptions

```
try:
    x = int(input())

    print(100/x)
```

```
except ZeroDivisionError:
    print("Cannot divide by zero")

except ValueError:
    print("Invalid integer")
```

8. Multiple Exceptions Together

```
try:
    x = int(input())

    print(100/x)

except (ValueError, ZeroDivisionError):
    print("Input Error")
```

9. Using else

The else block runs only if no exception occurs.

```
try:
    x = int(input())

except ValueError:
    print("Wrong input")

else:
    print("Square =", x*x)
```

Input: 8

Output: Square = 64

10. Using finally

The finally block always executes.

```
try:
    file = open("marks.txt")

except FileNotFoundError:
    print("File missing")

finally:
    print("Program finished")
```

Even if an exception occurs, the finally block runs.

Example

```
try:
    file = open("data.txt")

    number = int(file.readline())

    print(100/number)

except FileNotFoundError:
    print("File missing")

except ValueError:
    print("Invalid data")

except ZeroDivisionError:
    print("Zero found")

finally:
    print("Closing program")
```

11. Raising Exceptions

You can deliberately generate exceptions.

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative")
```

Output: ValueError: Age cannot be negative

12. Custom Exceptions

```
class NegativeAgeError(Exception):
    pass
```

Use it

```
age = -4

if age < 0:
    raise NegativeAgeError("Invalid Age")
```

Another example

```
class InsufficientBalance(Exception):
    pass

balance = 500

withdraw = 700

if withdraw > balance:
    raise InsufficientBalance("Low Balance")
```

13. Assertions

Assertions help detect programming mistakes during development.

```
x = 10

assert x > 0
```

No output.

Example

```
age = -10

assert age >= 0
```

Output: AssertionError

14. Exception Chaining

```
try:
    int("abc")

except ValueError as e:
    raise RuntimeError("Conversion failed") from e
```

This preserves the original exception context.

15. Common Programming Bugs

Off-by-one

```
for i in range(1,11):
```

vs

```
for i in range(10):
```

Infinite Loop

```
while True:
   print("Hello")
```

Integer Division Mistake

```
average = total // count
```

Should be

```
average = total / count
```

Wrong Variable

```
total = 100

discount = 10

print(total-total)
```

Should be

```
print(total-discount)
```

16. Debugging

Debugging means finding and fixing bugs.

Typical process:

1. Observe the symptom.
2. Reproduce the issue.
3. Isolate the cause.
4. Fix the code.
5. Retest.
6. Add tests to prevent regressions.

Method 1: Print Statements

```
a = 10
b = 0

print(a)
print(b)

print(a/b)
```

Method 2: Inspect Variables

```
numbers = [10,20,30]

print(len(numbers))
print(numbers)
```

Method 3: Trace Execution

```
for i in range(5):

    print("Loop", i)

print("Finished")
```

17. Using the Python Debugger (pdb)

```
import pdb

a = 10
b = 5

pdb.set_trace()

c = a + b

print(c)
```

Common pdb commands:

Command	Meaning
N	Next line
S	Step into function
C	Continue
p var	Print variable
L	List source code
Q	Quit debugger